DO NOW — RECALL

# C# Login Project · Complete and Test

Name: _____ Class: _____ Date: _____

**1. In a seven-bucket hash table, a collision occurs at bucket 6. Which bucket is checked next by linear probing, and why?**

**2. Predict the output of Console.WriteLine((age >= 16) && hasPermission); when age = 17 and hasPermission = false. Explain.**

**3. In a binary search tree, a new key is greater than the root but smaller than the root's right child. Where will it go if that right child has no left child?**

**4. A print server must process jobs in the order submitted. Choose a suitable data structure and explain why a stack would be unsuitable.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

Bucket 0: (6 + 1) MOD 7 = 0, so probing wraps around.

### 2. Model answer

False. The age test is true but hasPermission is false, so the AND expression is false.

### 3. Model answer

It becomes the left child of the root’s right child.

### 4. Model answer

Use a queue: FIFO serves the earliest job first. A stack uses LIFO and would process the most recently added job first.

### By the end, you can…

• Complete a classroom account-register and login system using the hash table.

• Test collisions, duplicates, successful and unsuccessful login attempts.

• Answer and improve student-written MCQs across OCR data structures.

### Vocabulary — know these terms

validation — Confirm data meets input rules.

duplicate — A repeated key that must not be inserted twice.

boundary test — Check a value at a limit, such as a full table.

erroneous test — Use invalid inputs to check handling.

linear probing — Search sequential buckets for an available place.

authentication — Check whether supplied credentials should grant access.

synoptic — Combining knowledge from multiple specification areas.

distractor — An incorrect but plausible multiple-choice option.

NEW LEARNING

# Read, annotate and explain

### Validation, testing and synoptic MCQs

Classroom demonstration only. Never type, collect or store real passwords.

### Your goal: a reliable demonstration system

Students will join registration and lookup so the program recognises existing usernames, refuses duplicates and performs login checks.

### Validation before storage

Reject blank usernames, duplicates and registrations when all buckets are full. Show a meaningful reason rather than crashing.

### Separate locating an account from checking a password

First use the hash table to find the username. Then compare an invented classroom demo password. These are separate operations.

### Design the test evidence

Test normal, erroneous and boundary scenarios: new user, unknown user, duplicate username, collision, table full and maximum attempts.

### Security boundary

The simple hash function here is unsuitable for real authentication. Production systems need established salted password-hashing libraries and secure storage.

### Synoptic retrieval

Exchange MCQs, answer independently, discuss distractors, then correct any technical errors. Cover the full OCR data-structures checklist.

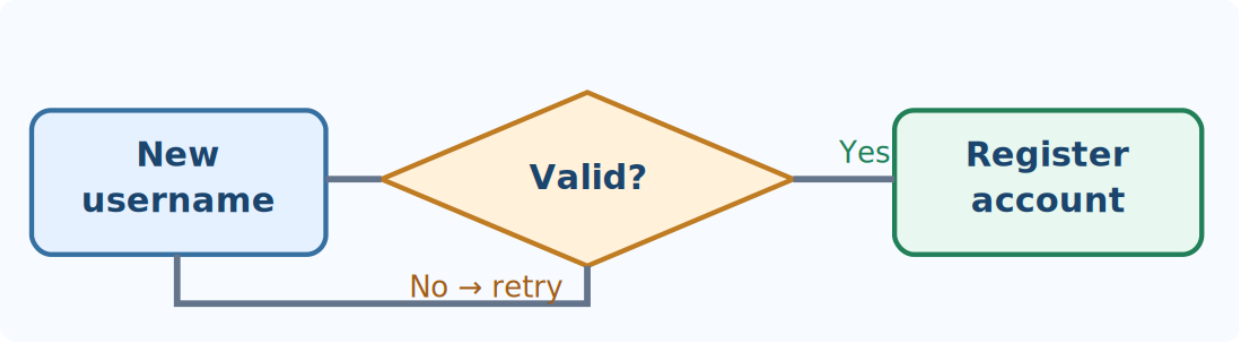
**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### Validation, testing and synoptic MCQs

Classroom demonstration only. Never type, collect or store real passwords.




Registration validation flow with a retry on invalid input

I DO — WORKED EXAMPLE

# Follow the worked method

## Model validation before registration

### Check blank input

```
Console.Write("New username: ");
string username = (Console.ReadLine() ?? "").Trim();
if (username.Length == 0)
{
    Console.WriteLine("Username required");
}
```

### Check whether the username exists

Use the lookup function from Lesson 033. Do not register duplicates, including usernames moved by probing.

### Enforce a size limit

Track how many table buckets are occupied. If all seven are full, refuse a new account gracefully.

WE DO — GUIDED PRACTICE

# Try → Discuss → Improve

### 1. Protect against duplicates

Which checks must happen before insertion?

Method / working:

### 2. Test a collision

How do we prove the collision handler works?

Method / working:

### 3. Build a synoptic MCQ

Which structure supports FIFO processing? Provide three plausible alternatives and justify the correct response.

Method / working:

# WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

### Guided 1: Protect against duplicates Which checks must happen before insertion? Explain your solution first

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

---

### Guided 2: Test a collision How do we prove the collision handler works?

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

---

### Guided 3: Build a synoptic MCQ Which structure supports FIFO processing? Provide three plausible alternatives and justify the correct response.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

---

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

## Question 1

CODING 1 · COMPLETE REGISTER
Extend the Lesson 033 table with registration, blank-name checks, duplicate-name rejection and full-table handling. Keep existing hash and lookup functions.

## Question 2

CODING 2 · COMPLETE LOGIN
Locate a registered username using the hash table. Then check an invented demonstration password associated with that account. Reject unknown usernames. Never enter real credentials.

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

CODING 3 · TEST TABLE
Evidence at least six tests: successful login, wrong demo password, unknown username, duplicate, collision, full-table boundary. Record input, expected, actual and pass/fail.

### Question 4

SYNOPTIC MCQ ASSESSMENT
Exchange student-written MCQs and answer independently. Cover arrays up to 3D, records, lists, tuples, linked lists, directed/undirected graphs, stacks, queues, trees, BSTs and hash tables. Identify and correct ambiguous distractors.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**Two usernames producing the same hash index must always overwrite one another.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

# WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

### Guided 1 — original CHECK

Protect against duplicates

CODE:

```
if (username.Length == 0)

Console.WriteLine("Username required");

else if (FindUser(username) >= 0)

Console.WriteLine("Username exists");

else

Console.WriteLine("Ready to register");
```

IMPROVE: Explain one correction to your own reasoning.

### Guided 2 — original CHECK

Explain your solution first

How do we prove the collision handler works?

Test a collision

Register 'ab' and 'ba'. Check both receive distinct buckets and FindUser locates both.

IMPROVE: Explain one correction to your own reasoning.

### Guided 3 — original CHECK

Explain your solution first

Which structure supports FIFO processing? Provide three plausible alternatives and justify the correct response.

Build a synoptic MCQ

Queue is correct. A stack is LIFO; a binary search tree organises ordered keys; a hash table is optimised for key-based lookup.

IMPROVE: Explain one correction to your own reasoning.